

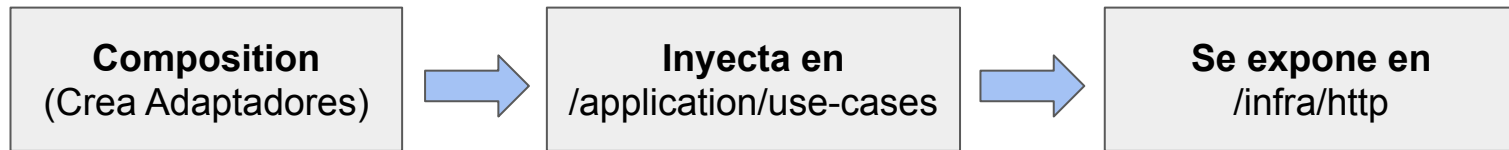
MÓDULO 2: ARQUITECTURA DE SOFTWARE

COMPOSICIÓN E INVERSIÓN DE DEPENDENCIAS

APLICANDO CLEAN ARCHITECTURE CON TYPESCRIPT

BIG school

Composición e inversión de dependencias



Principios de DI en Clean

- Inversión de Dependencias
- Composition Root
- Lifetimes
- Sin magia

Enrutando diferentes entornos (dev/test/prod)

DEV	<code>USE_INMEMORY=true</code> , logs verbosos, sin outbox real.
TEST	<code>USE_INMEMORY=true</code> o DB efímera + <code>TRUNCATE</code> entre tests.
PROD	DB real, outbox + dispatcher, timeouts estrictos.

Checklist de calidad de la composición

- ✓ Composition root único y visible.
- ✓ Ningún import de `infra` dentro de `application/domain`.
- ✓ Configuración **tipada** (fallar rápido).
- ✓ Lifetimes claros (qué es singleton y por qué).
- ✓ Tests “smoke” y de contrato corren en CI.

IA como Copiloto

Auditar el wiring

“Revisa este `container.ts` y señala dependencias cíclicas o singletons innecesarios. Propón lifetimes adecuados (singleton/scoped/transient).”

Generar composición por entorno

“Dado este `Config` y estos adaptadores, genera una función `buildAdapters(config)` que elija implementaciones por entorno y devuelva tipos concretos.”

Diseñar el scope por petición

“Crea un helper `makeRequestScope(container)` que inyecte `requestId`, `logger` con enlazado de contexto y ejemplos de uso en un controlador Fastify.”

Comprobación de límites

“Analiza estos imports y marca violaciones de la regla de dependencias entre `domain`, `application`, `infrastructure`, `composition`. Sugiere cambios y reglas ESLint.”